PROPOSAL TUGAS AKHIR

Pengembangan Agentic AI dengan Sistem Multi-Agen Berbasis LLM untuk Otomasi Pengujian Keamanan Web Berdasarkan Standar OWASP WSTG 4.2 Menggunakan Model Context Protocol

Martua Raja Doli Pangaribuan
2221101809

Rekayasa Keamanan Siber
Politeknik Siber dan Sandi Negara
2026

PROPOSAL TUGAS AKHIR

Diajukan untuk memenuhi persyaratan menyelesaikan Mata Kuliah Seminar Penulisan Tugas Akhir pada Politeknik Siber dan Sandi Negara

Pengembangan Agentic AI dengan Sistem Multi-Agen Berbasis LLM untuk Otomasi Pengujian Keamanan Web Berdasarkan Standar OWASP WSTG 4.2 Menggunakan Model Context Protocol

Martua Raja Doli Pangaribuan
2221101809

Rekayasa Keamanan Siber
Politeknik Siber dan Sandi Negara
2026

LEMBAR PERSETUJUAN

Proposal Tugas Akhir dengan:

JUDUL	:	Pengembangan Agentic AI dengan Sistem Multi-Agen Berbasis LLM untuk Otomasi Pengujian Keamanan Web Berdasarkan Standar OWASP WSTG 4.2 menggunakan Model Context Protocol
PENULIS	:	Martua Raja Doli Pangaribuan
NPM	:	2221101809

dinyatakan telah sesuai dengan Berita Acara Seminar Proposal Tugas Akhir dan disetujui untuk digunakan sebagai syarat kelulusan Mata Kuliah Seminar Penulisan Tugas Akhir Politeknik Siber dan Sandi Negara Angkatan 2022 tahun 2026.

Bogor, 21 Januari 2026

Pembimbing Materi

Girinoto, S.Si., M.Si
NIP. 197701092005011006

Dokumen ini telah ditandatangani secara elektronik menggunakan sertifikat elektronik yang diterbitkan oleh Balai Sertifikasi Elektronik (BSrE), Badan Siber dan Sandi Negara

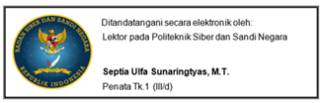
LEMBAR PENGESAHAN

Proposal Tugas Akhir dengan:

JUDUL	:	Pengembangan Agentic AI dengan Sistem Multi-Agen Berbasis LLM untuk Otomasi Pengujian Keamanan Web Berdasarkan Standar OWASP WSTG 4.2 menggunakan Model Context Protocol
PENULIS	:	Martua Raja Doli Pangaribuan
NPM	:	2221101809

telah diperiksa dan disahkan oleh Tim Penguji Seminar Proposal Tugas Akhir di Bogor, pada tanggal *tanggal bulan tahun sidang*

Ketua Penguji



Penguji I

Penguji II

Dokumen ini telah ditandatangani secara elektronik menggunakan sertifikat elektronik yang diterbitkan oleh Balai Sertifikasi Elektronik (BSrE), Badan Siber dan Sandi Negara

ABSTRAK

Pengembangan Agentic AI dengan Sistem Multi-Agen Berbasis LLM untuk Otomasi Pengujian Keamanan Web Berdasarkan Standar OWASP WSTG 4.2 Menggunakan Model Context Protocol

oleh

Martua Raja Doli Pangaribuan
2221101809

Rekayasa Keamanan Siber

Keamanan aplikasi web merupakan aspek kritis dalam ekosistem digital modern. Penelitian ini mengembangkan prototipe agen AI berbasis *Large Language Model* (LLM) yang terintegrasi dengan *vulnerability assessment tools* melalui *Model Context Protocol* (MCP), mengacu pada OWASP *Web Security Testing Guide* (WSTG) v4.2. Sistem dirancang untuk mengatasi tiga permasalahan utama yaitu fragmentasi alat pengujian keamanan, kompleksitas operasional yang tinggi, dan inkonsistensi hasil pengujian.

ix + 30 halaman (2026)
Kata kunci : *Model Context Protocol* (MCP), *Large Language Model* (LLM), *Vulnerability Assesment* (VA), Keamanan Aplikasi Web

DAFTAR ISI

Halaman
LEMBAR PENGESAHAN iv
ABSTRAK v
DAFTAR ISI vi
DAFTAR GAMBAR vii
DAFTAR TABEL viii
BAB I PENDAHULUAN 1
I.1 LATAR BELAKANG 1
I.2 RUMUSAN MASALAH 3
I.3 PEMBATASAN MASALAH 3
I.4 TUJUAN DAN MANFAAT 4
I.5 SISTEMATIKA PENULISAN 4
BAB II KAJIAN PUSTAKA 6
II.1 LANDASAN TEORI 6
II.2 PENELITIAN TERKAIT 13
II.3 KERANGKA KONSEPTUAL 17
BAB III METODOLOGI PENELITIAN 18
III.1 OBJEK PENELITIAN 18
III.2 JENIS PENELITIAN 18
III.3 DESAIN PENELITIAN 18
III.4 JADWAL PENELITIAN 25
DAFTAR PUSTAKA 26
DAFTAR RIWAYAT HIDUP 31

DAFTAR GAMBAR

Halaman

Gambar 1 Alur Uji Penetrasi 7
Gambar 2 Kerangka Konseptual 17

DAFTAR TABEL

Halaman

Tabel 1 Penelitian Terkait 14

BAB I
PENDAHULUAN

I.1 LATAR BELAKANG

Keamanan sistem digital masih menjadi tantangan utama di tengah keterhubungan layanan publik dan bisnis. Berdasarkan Lanskap Keamanan Siber Indonesia 2024 yang dirilis oleh Badan Siber dan Sandi Negara Republik Indonesia (BSSN), melaporkan temuan celah keamanan pada aplikasi web, mobile, serta infrastruktur yang dimiliki oleh instansi pemerintah pusat, pemerintah daerah, dan sektor Infrastruktur Informasi Vital (IIV). Dari jumlah tersebut, 256 celah tergolong *Critical*, 405 termasuk dalam kategori *High*, dan 350 diklasifikasikan sebagai *Medium* berdasarkan tingkat risikonya. Hal ini menunjukkan aplikasi web tetap menjadi permukaan serangan yang perlu diprioritaskan [1]. Di level global, laporan tahunan HackerOne 2024 mencatat sebanyak 78.042 laporan kerentanan yang valid, meningkat 12% dibanding tahun sebelumnya. Dari jumlah tersebut, 88% menargetkan aplikasi web, menjadikannya aset paling rentan dibandingkan aset lainnya [2]. Tingginya angka kerentanan ini berdampak langsung pada operasional bisnis seperti kerugian finansial akibat kerusakan sistem dan kehilangan kepercayaan dari pelanggan yang mendorong peningkatan pengujian keamanan, namun terkendala biaya tinggi dan keterbatasan sumber daya ahli.

Di saat kebutuhan otomasi pertahanan meningkat, laporan State of Cybersecurity 2024 dari ISACA menyoroti bahwa penggunaan A*rtificial Intelligence* (AI) dalam operasi keamanan siber masih sangat terbatas. Hanya 28% organisasi yang menggunakan AI untuk otomatisasi deteksi ancaman, 27% untuk keamanan *endpoint* dan 24% untuk otomasi tugas rutin. Selain itu, sebagian profesional keamanan siber belum terlibat dalam pengembangan atau implementasi AI. [3]. Rendahnya adopsi AI dalam keamanan siber ini membuka peluang untuk mengembangkan solusi yang dapat menjembatani kesenjangan antara kebutuhan pengujian keamanan yang komprehensif dengan keterbatasan sumber daya manusia yakni dengan otomatisasi.

Salah satu pendekatan yang banyak digunakan untuk mengidentifikasi celah keamanan secara proaktif adalah Uji Penetrasi (*penetration testing)*, yaitu *metode offensive security* yang bertujuan untuk mensimulasikan serangan nyata terhadap suatu sistem. Uji penetrasi umumnya terdiri dari lima fase: pengintaian, pemindaian, penilaian kerentanan, eksploitasi, dan pelaporan [4]. Praktik terkini banyak memanfaatkan alat otomatis seperti SQLmap, Dalfox, dan OWASP ZAP, namun penggunaannya masih tersebar dalam bentuk antarmuka baris perintah (*command line interface*) dengan sintaks beragam, sehingga memerlukan pemahaman teknis lebih agar maksimal yang tentunya menyulitkan pengguna non-teknis. [5][6].

Seiring berkembangnya teknologi, integrasi LLM dalam uji penetrasi menjadi topik yang kian dieksplorasi. Penelitian terbaru mengevaluasi pemanfaatan *prompt engineering*, *fine-tuning*, dan *Retrieval Augmented Generation* (RAG) untuk mengoperasikan serta menginterpretasi keluaran alat keamanan melalui bahasa alami. [7]. Pendekatan ini membuka potensi penggunaan LLM sebagai asisten cerdas dalam mengoperasikan dan menginterpretasi hasil alat pengujian keamanan. Di sisi lain, PentestGPT memperkenalkan arsitektur dengan tiga komponen utama yaitu *reasoning*, *generation*, dan *parsing*. Sistem ini mendemonstrasikan peningkatan performa task completion sebesar 228.6% dibandingkan dengan *baseline* GPT-3.5. Meskipun demikian, penelitian tersebut mengidentifikasi beberapa limitasi, termasuk kehilangan konteks (*context loss*) dan kebutuhan akan intervensi manusia dalam menangani skenario pengujian yang kompleks [8]. Pendekatan berbeda dari HackSynth yang mengimplementasikan arsitektur *dual-module* terdiri dari komponen *Planner* dan *Summarizer* untuk melakukan uji penetrasi otomatis pada tantangan *Capture The Flag* (CTF). Namun demikian, sistem ini memiliki keterbatasan dalam hal eksekusi yang masih bersifat sequential dan belum secara spesifik menargetkan pengujian keamanan aplikasi web [9]. Sementara itu, PENTEST-AI mengimplementasikan arsitektur multi-agen dengan pembagian peran spesifik untuk *scanning*, *exploit*, *validation*, dan *reporting,* serta mengintegrasikan basis pengetahuan MITRE ATT&CK untuk memandu proses pengujian [10]. Meskipun sistem ini mendemonstrasikan pendekatan multi-agen dengan tahapan yang terstruktur, kerangka MITRE ATT&CK yang digunakan bersifat umum dan tidak secara langsung mencerminkan prosedur uji yang dibutuhkan untuk pengujian keamanan aplikasi web.

Rangkaian temuan tersebut menegaskan adanya kesenjangan yang perlu dijawab. Meskipun PENTEST-AI telah mengimplementasikan arsitektur multi-agen untuk otomasi uji penetrasi, kerangka evaluasi MITRE ATT&CK yang digunakan bersifat umum dan tidak secara spesifik mencerminkan prosedur pengujian keamanan aplikasi web. PentestGPT masih menggunakan pendekatan single-agent yang rentan terhadap kehilangan konteks, sementara HackSynth berfokus pada tantangan CTF dengan eksekusi sequential. Berdasarkan tinjaun literatur yang dilakukan, belum ditemukan implementasi yang secara spesifik mengintegrasikan arsitektur multi-agen dengan standar OWASP WSTG versi 4.2 melalui protokol Model Context Protocol (MCP) untuk pengujian keamanan aplikasi web secara sistematis.

Penelitian ini memfokuskan evaluasi pada pendekatan kecerdasan agenik dengan arsitektur multi-agen terspesialiasi untuk penilaian kerentanan aplikasi web. Setiap agen memiliki keahlian yang selaras dengan kategori dalam yang terdapat OWASP Web Security Testing Guide versi 4.2 yaitu *Information Gathering*, *Configuration and Deployment Management Testing*, *Identity Management Testing*, *Authentication Testing*, *Authorization Testing*, *Session Management Testing*, *Input*

Validation Testing, *Testing for Error Handling*, *Testing for Weak Cryptography*, *Business Logic Testing* dan *Client Side Testing [11]*. OWASP WSTG dipilih sebagai kerangka evaluasi karena menyediakan daftar uji yang rinci dan komprehensif dan relevan khusus untuk aplikasi web.[12]

Untuk menyederhanakan integrasi dengan alat uji, penelitian ini memanfaatkan *Model Context Protocol* (MCP) yang diperkenalkan pada 2024. Protokol ini menempatkan LLM sebagai klien yang berkomunikasi dengan penyedia alat dan sumber data melalui JSON-RPC 2.0 sehingga perintah pemindaian atau eksploitasi dapat disalurkan dengan mekanisme yang konsisten [13]. Selain itu, OWASP Juice Shop digunakan sebagai *benchmark dataset* karena memiliki tantangan keamanan terstandarisasi yang sesuai dengan kategori OWASP WSTG 4.2 dan telah digunakan dalam penelitian terkini. [14]

Berdasarkan latar belakang tersebut, penelitian ini bertujuan untuk mengembangkan sistem multi-agen berbasis Large Language Model (LLM) yang mampu mengotomatisasi pengujian keamanan aplikasi web secara menyeluruh berdasarkan standar OWASP Web Security Testing Guide (WSTG) versi 4.2. Penelitian ini juga bertujuan untuk mengevaluasi efektivitas sistem yang dikembangkan dalam mendeteksi kerentanan secara valid, dengan tingkat positif palsu yang rendah, serta efisiensi pelaksanaan pengujian. Penilaian dilakukan dengan mempertimbangkan indikator kinerja seperti cakupan pengujian (completion rate), menguji fungsi-fungsi teknis pada sistem, menguji performa sistem dan menguji kelayakan sistem apakah layak atau tidaknya digunakan.

I.2 RUMUSAN MASALAH

Berdasarkan latar belakang yang telah diuraikan, rumusan permasalahan dalam penelitian Tugas Akhir ini, yaitu:

a. Bagaimana tingkat akurasi sis kerentanan yang dihasilkan oleh sistem multi-agen berbasis LLM saat diuji menggunakan OWASP Juice Shop berdasarkan standar OWASP Web Security Testing Guide (WSTG) v4.2, serta diukur menggunakan metrik Precision, Recall, dan F1-Score (F-Measure)?
b. Bagaimana kinerja dan usabilitas sistem yang dikembangkan, ditinjau dari Task Completion Rate (TCR dan System Usability Scale (SUS)?

I.3 PEMBATASAN MASALAH

Dalam penelitian Tugas Akhir ini, terdapat beberapa pembatasan yang digunakan yaitu meliputi :

a. Penelitian ini difokuskan pada pengujian keamanan aplikasi web berdasarkan kategori dan panduan OWASP Web Security Testing Guide (WSTG) v4.2.

b. Target pengujian menggunakan aplikasi web dummy OWASP Juice Shop, yang secara resmi dirancang OWASP sebagai platform uji kerentanan

c. Pengujian keamanan hanya menggunakan tools open-source tertentu yang terintegrasi melalui Model Context Protocol

d. Evaluasi sistem berfokus pada metrik internal seperti Task Completion Rate (TCR), Precision, Recall, dan F1-Score, serta membandingkan hasilnya dengan baseline dari penelitian terdahulu.

I.4 TUJUAN DAN MANFAAT

I.4.1 Tujuan

Tujuan dari penelitian ini yaitu:

a. Mengembangkan dan mengevaluasi sistem multi-agen berbasis Large Language Model (LLM) yang mampu mengotomatisasi proses pengujian keamanan aplikasi web sesuai standar OWASP Web Security Testing Guide (WSTG) v4.2, dengan memanfaatkan Model Context Protocol (MCP) sebagai kerangka komunikasi antarkomponen untuk integrasi alat uji open-source.

b. Menguji efektivitas dan akurasi sistem dalam mendeteksi kerentanan keamanan aplikasi web menggunakan OWASP Juice Shop sebagai benchmark dataset, dengan evaluasi berbasis metrik kuantitatif seperti Precision, Recall, F1-Score (F-Measure), serta metrik tambahan Task Completion Rate (TCR), False Positive Rate (FPR), Average Time per Test (ATT), dan System Usability Scale (SUS).

I.4.2 Manfaat

Penelitian ini diharapkan dapat bermanfaat bagi seluruh pihak. Beberapa manfaat yang diharapkan di antaranya adalah:

a. Menambah kontribusi ilmiah di bidang keamanan siber melalui pengembangan agentic AI berbasis LLM yang terintegrasi dengan vulnerability assessment tools melalui Model Context Protocol (MCP) dan standar OWASP WSTG v4.2.

b. Mempermudah praktisi keamanan dalam melakukan uji penetrasi terhadap aplikasi web tanpa membutuhkan pemahaman teknis mendalam terhadap alat uji.

c. Memperkuat posisi Politeknik Siber dan Sandi Negara sebagai institusi yang mendukung riset inovatif dalam bidang pertahanan siber nasional.

I.5 SISTEMATIKA PENULISAN

Penulisan Proposal ini terbagi menjadi tiga bab yang saling berkaitan membentuk satu kesatuan dengan sistematika penulisan sebagai berikut:

BAB I : PENDAHULUAN

Bab ini membahas mengenai latar belakang masalah, rumusan masalah, pembatasan masalah, tujuan dan manfaat, serta sistematika penulisan.

BAB II : KAJIAN PUSTAKA

Bab ini membahas konsep-konsep dasar, teori, dan hasil penelitian terdahulu yang berkaitan dengan *Penetration Testing*, *Large Language Model* (LLM), *Model Context Protocol* (MCP), serta *OWASP Web Security Testing Guide* (WSTG) v4.2 sebagai standar acuan dan metrik evaluasi.

BAB III : METODOLOGI PENELITIAN

Bab ini membahas metodologi penelitian yang digunakan, meliputi desain sistem, tahapan implementasi agen AI berbasis LLM dan MCP, skenario pengujian menggunakan *checklist* OWASP WSTG v4.2, serta metode evaluasi kinerja sistemnya

BAB II KAJIAN PUSTAKA

II.1 LANDASAN TEORI

Pada bagian ini akan membahas mengenai konsep-konsep dan teori-teori yang mendukung pelaksanaan penelitian, meliputi *Penetration Testing*, OWASP *Web Security Testing Guide* (WSTG), *Large Language Model* (LLM), *Model Context Protocol* (MCP) dan metrik evaluasi.

II.1.1 Keamanaan Aplikasi Web

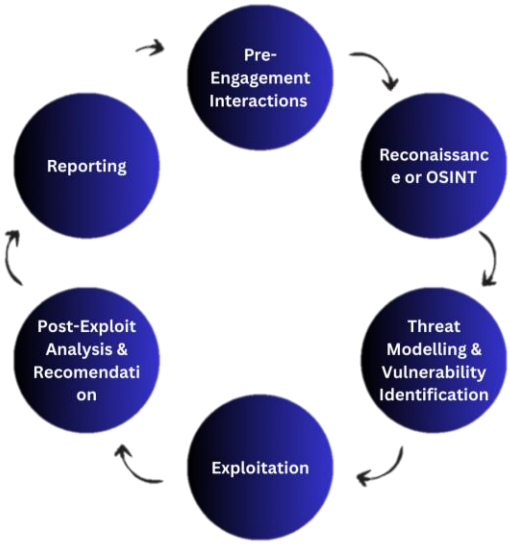
Keamanan aplikasi web merupakan salah satu aspek fundamental dalam pengembangan dan operasionalisasi sistem berbasis internet. Secara umum, keamanan aplikasi web dapat didefinisikan sebagai upaya sistematis untuk melindungi aplikasi dari ancaman yang berpotensi mengganggu kerahasiaan (confidentiality), integritas (integrity), dan ketersediaan (availability) dari data dan layanan yang disediakan aplikasi tersebut [15]. Menurut Open Web Application Security Project (OWASP), keamanan aplikasi web berfokus pada pencegahan dan mitigasi berbagai jenis serangan yang menargetkan lapisan aplikasi, terutama pada interaksi antara pengguna, server, dan basis data.[16], [17]

Dalam konteks digitalisasi nasional dan implementasi Sistem Pemerintahan Berbasis Elektronik (SPBE) sebagaimana diatur dalam Peraturan Presiden Nomor 82 Tahun 2023, keamanan aplikasi web menjadi krusial karena mayoritas layanan publik kini diakses melalui platform daring. Celah keamanan sekecil apa pun dapat berdampak besar, mulai dari kebocoran data pribadi, defacement situs pemerintah, hingga pengambilalihan kendali sistem. Berdasarkan laporan Badan Siber dan Sandi Negara, serangan terhadap aplikasi web menempati peringkat tiga besar insiden siber terbanyak di Indonesia, terutama pada sektor layanan publik dan pendidikan. [18]

Ancaman terhadap aplikasi web umumnya muncul akibat kelemahan dalam mekanisme autentikasi, validasi input, dan konfigurasi sistem. Jenis serangan yang paling sering ditemukan meliputi SQL Injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), Broken Authentication, dan Server-Side Request Forgery (SSRF). OWASP secara berkala merilis OWASP Top 10 yang berisi daftar sepuluh kategori risiko keamanan aplikasi web paling kritis. Versi terbarunya, OWASP Top 10 2025, menekankan bahwa lebih dari 80% kerentanan berasal dari kelemahan validasi input dan pengelolaan sesi (session management), yang dapat dieksploitasi dengan teknik sederhana namun berdampak tinggi.[19]

II.1.2 Uji Penetrasi

Uji penetrasi (penetration testing) merupakan salah satu metode penting dalam peningkatan keamanan sistem dengan cara mensimulasikan serangan untuk menemukan celah yang dapat dieksploitasi [20]. Menurut National Institute of Standards and Technology (NIST), uji penetrasi adalah bentuk penilaian khusus yang dilakukan terhadap sistem informasi atau komponen tertentu untuk mengidentifikasi kerentanan yang berpotensi dimanfaatkan oleh pihak tidak berwenang [21]. Tujuan utama pentest adalah mengevaluasi sistem, mendeteksi potensi kelemahan, dan memberikan rekomendasi langkah perbaikan guna memperkuat keamanan secara keseluruhan.



Gambar 1 Alur Uji Penetrasi

Individu atau tim yang melaksanakan uji penetrasi disebut pentester. Dalam praktiknya, seorang pentester berperan layaknya penyerang dan mengikuti serangkaian tahapan yang umumnya dibagi menjadi lima fase: pengintaian, pemindaian, penilaian kerentanan, eksploitasi, dan pelaporan [22]. Kelima fase tersebut saling berkaitan, dimulai dari:

1. **Pengintaian (*Reconnaissance*)**
   Mengumpulkan informasi awal terkait target, seperti nama domain utama, subdomain, alamat IP, firewall yang digunakan, port yang terbuka, dan data teknis lainnya [23]. Informasi ini menjadi landasan untuk tahapan berikutnya.
2. **Pemindaian (*Scanning*)**
   Menggunakan berbagai alat untuk mendapatkan detail tambahan seperti layanan aktif, versi perangkat lunak, sistem operasi, dan konfigurasi jaringan [24], [25].
3. **Pencarian Kerentanan (*Discover Vulnerability*)**
   Memanfaatkan hasil dua fase sebelumnya untuk mengidentifikasi celah keamanan yang ada.

4. **Eksploitasi (*Exploitation*)**
   Menguji celah yang ditemukan untuk membuktikan dampaknya terhadap keamanan sistem [26].
5. **Pelaporan (*Report Generation*)**
   Menyusun laporan temuan yang berisi jenis kerentanan, bukti pendukung, dan potensi risiko yang diakibatkan [27].

Terdapat tiga pendekatan yang digunakan dalam pentest berdasarkan tingkat informasi yang dimiliki penguji: gray box (informasi terbatas) [28], white box (informasi penuh) [29], dan black box (tanpa informasi awal) [30]. Pada pelaksanaannya, pentest memanfaatkan gabungan alat otomatis dan manual seperti Burp Suite, SQLmap, Nessus, atau Nuclei untuk mempermudah deteksi dan eksploitasi kerentanan. [24], [31]

Dengan jumlah pengguna internet yang pada 2024 telah mencapai sekitar 5,35 miliar [32], kebutuhan akan uji penetrasi pada sistem berbasis web semakin mendesak. Semakin luasnya pemanfaatan platform web secara langsung meningkatkan risiko serangan siber.

Dalam penelitian ini, uji penetrasi dilakukan dengan memanfaatkan serangkaian *open-source tools* yang terintegrasi, di antaranya:

a) **Whatweb**
   Merupakan alat *Web technology fingerprinting* (CMS, server, framework, plugin, email, versi, error SQL, dll.).[33]
b) **SQLmap**
   Merupakan alat untuk otomatisasi deteksi & eksploitasi SQLi hingga *DB takeover* (fingerprinting, data fetch, file/OS).[34]
c) **Dalfox**
   Merupakan alat untuk mengidentifikasi potensi *Cross-site Scripting (XSS)*.[35]
d) **Nmap**
   Merupakan alat untuk memindai jaringan untuk mendeteksi port, layanan, dan OS.[36]
e) **Whois**
   Merupakan alat untuk mendapatkan data pendaftar domain/IP (registrar, kontak, tanggal, dsb.).[37]
f) **Testssl**
   Merupakan alat untuk menguji TLS/SSL via CLI: protokol, cipher, dan kerentanan kriptografi pada layanan/port.[38]
g) **Nuclei**
   Merupakan alat untuk memindai target berdasarkan *template* kerentanan.[39]
h) **puppeter**

Merupakan pustaka JS untuk otomasi Chrome/Firefox (DevTools/WebDriver BiDi): navigasi, tangkapan layar, uji UI, dsb.[40]

II.1.3 Large Language Model (LLM)

Large Language Model (LLM) merupakan kelas model pembelajaran mesin yang dirancang untuk memahami, memproses, dan menghasilkan teks dalam bahasa alami. Model ini disebut "large" karena dilatih menggunakan miliaran hingga triliunan parameter pada korpus teks yang sangat besar. Arsitektur yang mendasari sebagian besar LLM modern adalah Transformer, yang diperkenalkan oleh Vaswani et al. pada tahun 2017 [41]. Berbeda dengan arsitektur sebelumnya seperti Recurrent Neural Network (RNN) yang memproses urutan secara sekuensial, Transformer menggunakan mekanisme self-attention yang memungkinkan model memproses seluruh input secara paralel dan menangkap hubungan jarak jauh antarkata dengan lebih efektif.

Secara umum, LLM bekerja dengan memprediksi token berikutnya berdasarkan token-token sebelumnya. Proses pelatihan terdiri dari dua tahap utama, yaitu pre-training pada korpus berskala besar untuk mempelajari pola bahasa, kemudian fine-tuning untuk menyesuaikan model pada tugas spesifik. Beberapa model juga menerapkan Reinforcement Learning from Human Feedback (RLHF) untuk menyelaraskan keluaran dengan preferensi manusia. Model-model mutakhir seperti GPT-4o (OpenAI), Claude 3.5 Sonnet (Anthropic), dan Llama 3.1 (Meta) telah mendemonstrasikan kemampuan penalaran kompleks, pemrograman, dan pemecahan masalah yang melampaui sekadar pemrosesan bahasa.[42]

Meskipun memiliki kemampuan yang mengesankan, LLM juga memiliki keterbatasan. Fenomena hallucination menyebabkan model dapat menghasilkan informasi yang tidak akurat. Keterbatasan context window membatasi jumlah teks yang dapat diproses dalam satu waktu. Sifat non-deterministic menghasilkan respons berbeda untuk input yang sama. Selain itu, knowledge cutoff membatasi pengetahuan model pada data pelatihan sehingga tidak mengetahui informasi terkini. Keterbatasan-keterbatasan ini menjadi pertimbangan penting dalam pengembangan aplikasi berbasis LLM, termasuk dalam domain keamanan siber.[43]

II.1.4 Sistem Multi-Agen

Sistem multi-agen (Multi-Agent System) merupakan paradigma dalam kecerdasan buatan yang terdiri dari beberapa entitas otonom, disebut agen, yang berinteraksi dan bekerja sama untuk mencapai tujuan tertentu. Wooldridge mendefinisikan agen sebagai sistem komputer yang memiliki kemampuan otonomi untuk beroperasi tanpa intervensi langsung manusia, serta kemampuan sosial untuk berinteraksi dengan agen lain [44]. Setiap agen dalam sistem dapat mengamati lingkungannya,

mengambil keputusan, dan melakukan tindakan secara independen. Pendekatan multi-agen memungkinkan penyelesaian masalah kompleks yang sulit ditangani oleh agen tunggal melalui pembagian tugas dan kolaborasi.

Perkembangan LLM telah membawa evolusi signifikan dalam sistem multi-agen. Jika sebelumnya komunikasi antaragen terbatas pada pertukaran simbolik dengan aturan kaku, kini agen berbasis LLM dapat berkomunikasi menggunakan bahasa alami dan melakukan penalaran kontekstual secara adaptif [45]. Integrasi ini melahirkan konsep Agentic AI, di mana setiap agen memiliki kemampuan memahami instruksi kompleks dan berkolaborasi secara dinamis.

Tantangan utama dalam sistem multi-agen berbasis LLM adalah sinkronisasi konteks antaragen karena setiap agen memiliki jendela konteks terbatas [46]. Untuk mengatasi hal ini, diperlukan mekanisme koordinasi yang dapat menjaga konsistensi konteks selama proses kolaborasi. Penelitian ini mengusulkan penggunaan Model Context Protocol (MCP) sebagai lapisan komunikasi yang memungkinkan pertukaran informasi terstruktur antaragen tanpa kehilangan konteks pengujian.

II.1.5 *Model Context Protocol* (MCP)

Model Context Protocol (MCP) adalah standar terbuka yang diperkenalkan oleh Anthropic pada November 2024 untuk menstandarisasi cara sistem kecerdasan buatan mengintegrasikan dan berbagi data dengan alat eksternal serta sumber data [13].

MCP menggunakan arsitektur client-server yang terinspirasi dari Language Server Protocol (LSP) dan ditransportasikan melalui JSON-RPC 2.0. Dalam arsitektur ini, MCP Host adalah aplikasi yang menggunakan LLM seperti Claude Desktop atau IDE, MCP Client adalah komponen yang menangani komunikasi protokol, dan MCP Server adalah layanan yang menyediakan akses ke tools atau sumber data. Spesifikasi MCP mendefinisikan tiga primitif server yaitu Tools untuk fungsi yang dapat dieksekusi, Resources untuk data terstruktur yang dapat dimasukkan ke konteks LLM, dan Prompts untuk template instruksi. Selain itu terdapat dua primitif client yaitu Roots untuk akses filesystem dan Sampling untuk memungkinkan server meminta completions dari LLM [47]. Dalam konteks penelitian ini, MCP berfungsi sebagai lapisan komunikasi yang memungkinkan agen-agen berbasis LLM untuk berkoordinasi dan berbagi konteks pengujian keamanan tanpa kehilangan informasi penting antaragen.

II.1.6 OWASP Web Security Testing Guide (WSTG) v4.2

OWASP *Web Security Testing Guide* (WSTG) adalah panduan terbuka yang disusun oleh komunitas *Open Worldwide Application Security Project* (OWASP) untuk melakukan pengujian keamanan aplikasi web secara menyeluruh. Edisi v4.2

mencakup lebih dari 90 skenario pengujian yang terbagi ke dalam 11 kategori [11]. Adapun 11 kategori pengujian sebagai berikut:

a) ***Information Gathering***
   Merupakan tahap untuk pengumpulan informasi awal (subdomain, *directory enumeration*, *server/banner*).
b) ***Configuration & Deployment Management Testing***
   Merupakan tahap untuk memeriksa konfigurasi server/aplikasi, file sensitif, header keamanan.
c) ***Identity Management Testing***
   Merupakan tahap untuk memeriksa pengelolaan identitas dan kebijakan akun.
d) ***Authentication Testing***
   Merupakan tahap untuk memeriksa mekanisme autentikasi (*password policy, multi-factor, brute-force protections*).
e) ***Authorization Testing***
   Merupakan tahap untuk memeriksa kontrol otorisasi (*role/permission*, IDOR, akses horizontal/vertikal).
f) ***Session Management Testing***
   Merupakan tahap untuk memeriksa manajemen sesi (*cookie flags, session fixation, session timeout*).
g) ***Input Validation Testing***
   Merupakan tahap untuk memeriksa validasi masukan (*XSS, SQLi, command injection, file upload*).
h) ***Error Handling Testing***
   Merupakan tahap untuk memeriksa pengungkapan informasi melalui *error/stack trace/logs.*
i) ***Cryptography Testing***
   Merupakan tahap untuk memeriksa praktik kriptografi (TLS/SSL, *storage/transport encryption*, *key management*).
j) ***Business Logic Testing***
   Merupakan tahap untuk memeriksa penyalahgunaan alur/proses bisnis yang valid *(logic flaws).*
k) ***Client-side Testing***
   Merupakan tahap untuk memeriksa keamanan sisi-klien (*DOM XSS*, *CORS*, *CSP*, *storage lokal*, dependensi *JS*).

II.1.7 Model Pengembangan Sistem

Model pengembangan sistem yang digunakan adalah Software Development Life Cycle (SDLC). SDLC merupakan metodologi yang digunakan untuk merancang dan membangun sistem perangkat lunak secara terstruktur. Model ini menjelaskan tahapan kerja mulai dari analisis kebutuhan hingga evaluasi hasil sistem yang telah dikembangkan [48].

Dalam penelitian ini digunakan model Waterfall karena pendekatan ini sesuai dengan karakteristik penelitian yang bersifat eksperimental dan membutuhkan alur kerja yang sistematis. Model Waterfall juga memungkinkan dokumentasi yang baik di setiap tahapan, sehingga mendukung proses validasi sistem multi-agen berbasis Large Language Model (LLM) dan Model Context Protocol (MCP) yang diusulkan. Tahapan dalam model ini terdiri atas beberapa langkah sebagai berikut:

a) Analisis Kebutuhan (Requirements)
   Tahap ini melibatkan identifikasi kebutuhan fungsional dan non-fungsional sistem. Kebutuhan fungsional mencakup perancangan agen LLM, mekanisme komunikasi MCP, serta integrasi dengan alat uji keamanan web. Kebutuhan non-fungsional mencakup efisiensi, keamanan, dan kesesuaian dengan standar OWASP WSTG v4.2.
b) Perancangan Sistem (System Design)
   Meliputi perancangan arsitektur multi-agen dan alur komunikasi MCP. Desain ini juga menentukan bagaimana data hasil pengujian akan disinkronkan antar agen.
c) Implementasi (Implementation)
   Pada tahap ini, sistem direalisasikan menggunakan bahasa pemrograman Python dan API LLM Setiap agen dikembangkan berdasarkan peran fungsionalnya dan diintegrasikan melalui MCP.
d) Pengujian (Testing)
   Dilakukan untuk memastikan bahwa setiap agen dan fungsi sistem bekerja sesuai harapan. Pengujian mencakup uji fungsional, uji integrasi, serta pengujian efektivitas sistem berdasarkan metrik Task Completion Rate (TCR) dan System Usability Scale (SUS).
e) Evaluasi dan Dokumentasi (Maintenance)
   Tahap akhir melibatkan analisis hasil pengujian dan dokumentasi teknis. Evaluasi dilakukan untuk menilai efektivitas sistem dalam mendeteksi kerentanan aplikasi web dan memastikan kesesuaiannya dengan standar OWASP WSTG v4.2.

Dalam penelitian ini, model SDLC tidak digunakan sebagai kerangka penelitian utama, tetapi sebagai model pengembangan teknis yang mendukung tahap Design and Development dari metodologi Design Science Research Methodology (DSRM) [49]. Dengan demikian, SDLC berperan dalam mengatur proses pembangunan artefak sistem mulai dari analisis kebutuhan, perancangan arsitektur multi-agen, implementasi sistem berbasis LLM dan Model Context Protocol (MCP), hingga tahap pengujian dan pemeliharaan.

II.1.8 Metode Pengujian dan Evaluasi

Evaluasi sistem yang dikembangkan dalam penelitian ini menggunakan dua kategori metrik, yaitu metrik efektivitas deteksi kerentanan dan metrik usabilitas sistem. Metrik efektivitas mengadopsi konsep dari bidang information retrieval

yang telah menjadi standar dalam evaluasi sistem deteksi dan klasifikasi [50]. Metrik usabilitas menggunakan instrumen terstandar yang telah divalidasi secara empiris untuk mengukur persepsi pengguna terhadap sistem [51]. Adapun kedua tahapannya sebagai berikut:

a) Evaluasi Efektivitas Deteksi Kerentanan
   Evaluasi efektivitas sistem dilakukan untuk menilai kemampuan sistem dalam mendeteksi kerentanan aplikasi web secara otomatis, akurat, dan efisien. Penilaian dilakukan menggunakan indikator utama Task Completion Rate (TCR) yaitu untuk mengukur persentase keberhasilan sistem dalam menyelesaikan seluruh task pengujian.

$$TCR = \frac{\text{Jumlah Test Case Selesai}}{\text{Total Test Case WSTG}} \times 100\%$$

b) Evaluasi Usability Sistem
   Usabilitas sistem dievaluasi menggunakan System Usability Scale (SUS), instrumen evaluasi terstandar yang dikembangkan oleh Brooke [51] dan telah divalidasi melalui berbagai studi empiris. SUS terdiri dari 10 pernyataan yang dijawab menggunakan skala Likert 5 poin, dengan pernyataan bernomor ganjil bersifat positif dan pernyataan bernomor genap bersifat negatif. Skor SUS dihitung dengan menjumlahkan kontribusi setiap item lalu mengalikannya dengan 2.5 untuk menghasilkan skor dalam rentang 0-100. Interpretasi skor SUS mengacu pada skala adjektif di mana skor di atas 68 dianggap di atas rata-rata, skor 68 merupakan rata-rata, dan skor di bawah 68 dianggap di bawah rata-rata [52].

Melalui tiga tahapan pengujian ini, penelitian diharapkan mampu menilai sistem secara menyeluruh, baik dari sisi keberfungsian teknis, efektivitas dalam mendeteksi kerentanan, maupun tingkat kenyamanan pengguna. Pendekatan ini memastikan bahwa sistem yang dikembangkan tidak hanya valid secara teknis, tetapi juga memiliki nilai praktis dalam penerapan pengujian keamanan aplikasi web secara otomatis sesuai standar OWASP WSTG v4.2.

II.2 PENELITIAN TERKAIT

Terdapat beberapa penelitian sebelumnya yang berkaitan dengan uji penetrasi. Berikut disajikan pembahasan dari *literature review* yang telah dilakukan penulis terhadap beberapa penelitian terkait.

Tabel 1 Penelitian Terkait

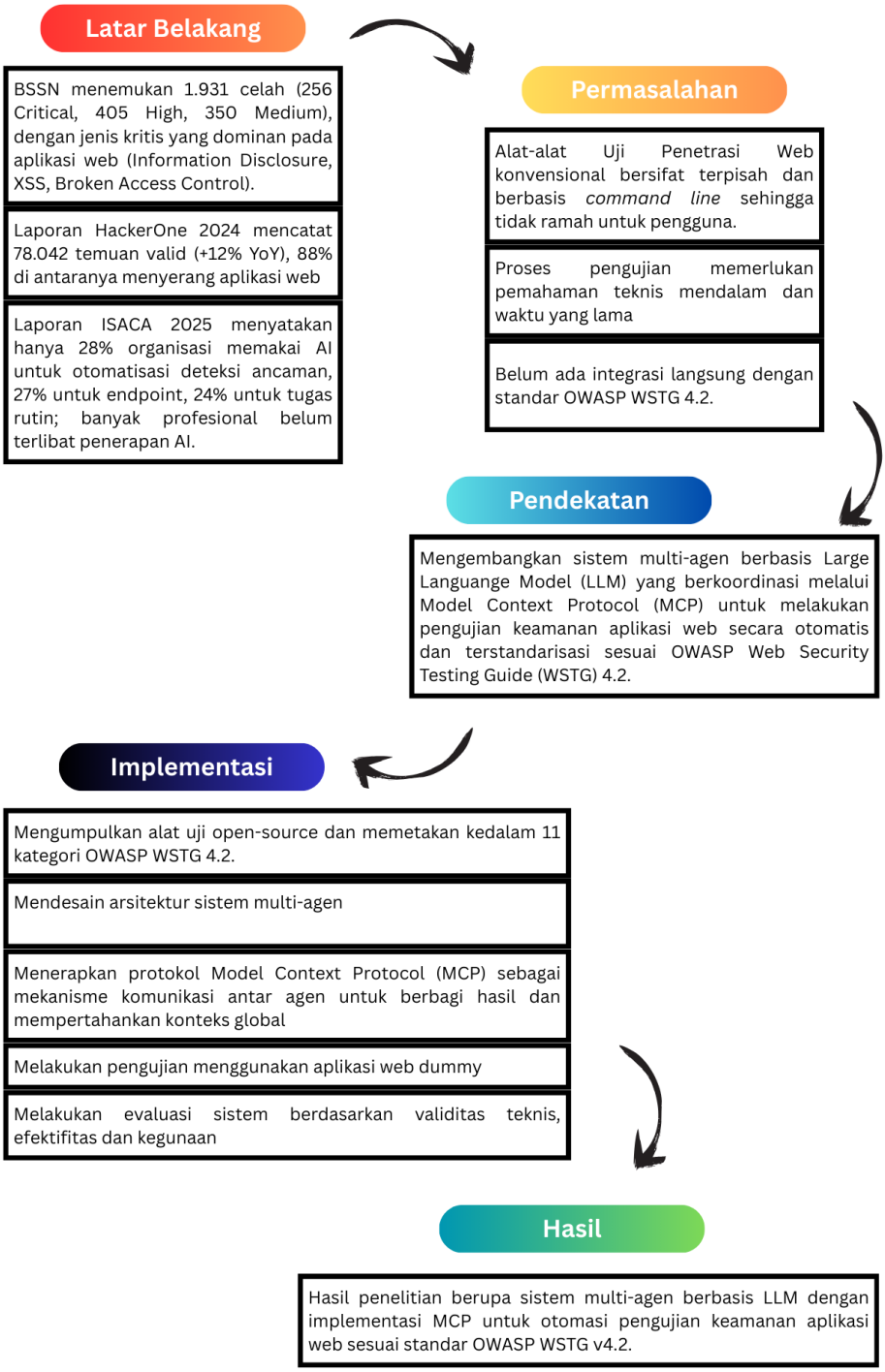
No	Peneliti dan Tahun Terbit	Topik Penelitian	Hasil Penelitian	Keterkaitan
1	Deng et al. (2024)[53]	PentestGPT: An LLM-empowered Automatic Penetration Testing Tool	Penelitian ini memperkenalkan PentestGPT, framework otomatisasi penetration testing berbasis LLM yang meniru kolaborasi tim penguji keamanan manusia melalui tiga modul utama: Reasoning, Generation, dan Parsing. Sistem ini diuji pada dataset HackTheBox dan VulnHub, menunjukkan peningkatan task completion rate sebesar 228,6% dibanding GPT-3.5, serta mampu menyelesaikan tantangan nyata pada picoMini CTF. Pendekatan ini menegaskan potensi LLM dalam melakukan semi-autonomous penetration testing dengan bimbingan manusia (human-in-the-loop).	Struktur modular sistem multi-agen, penggunaan task tree reasoning, serta evaluasi kinerja berbasis Task Completion Rate (TCR).
2	Vadla et Al (2024)[54]	Integrating Penetration Testing Tools into a Website	Penelitian ini mengusulkan integrasi berbagai alat uji penetrasi seperti Nmap, Nikto, SQLmap, dan Searchsploit ke dalam platform berbasis web untuk meningkatkan aksesibilitas dan efisiensi proses pengujian keamanan. Integrasi dilakukan melalui metode Backend Command Execution	Memberikan dasar empiris tentang integrasi alat uji keamanan ke dalam platform berbasis web. Relevan dengan penelitian ini karena menjadi pembanding bagi sistem multi-agen berbasis

No	Peneliti dan Tahun Terbit	Topik Penelitian	Hasil Penelitian	Keterkaitan
			yang memungkinkan alat CLI dijalankan dari antarmuka web, serta dibandingkan dengan pendekatan lain seperti API Integration dan Containerization. Penelitian ini menyoroti risiko keamanan baru seperti SQL Injection dan XSS akibat integrasi langsung, dan merekomendasikan pendekatan modular berbasis API sebagai solusi yang lebih aman.	LLM dan MCP yang juga mengintegrasikan open-source security tools namun dengan koordinasi kontekstual otomatis dan kontrol validasi hasil.
3	Muzsai et al. (2024)[55]	HackSynth: LLM Agent and Evaluation Framework for Autonomous Penetration Testing	Penelitian ini mengusulkan HackSynth, sistem autonomous penetration testing agent berbasis LLM yang mengadopsi dua modul inti, yaitu Planner dan Summarizer, untuk mengelola feedback loop eksploitasi dan validasi hasil. Pengujian dilakukan menggunakan dataset PicoCTF dan OverTheWire dengan lebih dari 200 tantangan keamanan. Hasil menunjukkan peningkatan completion rate dan efisiensi reasoning dibanding pendekatan manual. Selain itu, penelitian ini menekankan keamanan lingkungan pengujian melalui containerized isolation.	Arsitektur dua lapis Planner–Summarizer sebagai acuan peran antar agen, serta metode evaluasi berbasis Completion Rate dan Reliability.

No	Peneliti dan Tahun Terbit	Topik Penelitian	Hasil Penelitian	Keterkaitan

II.3 KERANGKA KONSEPTUAL

Kerangka konseptual menjelaskan hubungan variabel dan alur proses pada penelitian ini.



Gambar 2 Kerangka Konseptual

BAB III
METODOLOGI PENELITIAN

Bab ini membahas mengenai objek penelitian, jenis penelitian, desain penelitian, dan jadwal penelitian.

III.1 OBJEK PENELITIAN

Objek penelitian ini adalah pengembangan sistem multi-agen berbasis Large Language Model yang diorkestrasi melalui Model Context Protocol (MCP) untuk melakukan pengujian keamanan aplikasi web secara otomatis berdasarkan standwar OWASP Web Security Testing Guide (WSTG) versi 4.2. Sistem ini dirancang untuk mengintegrasikan beberapa alat uji keamanan website yang open source seperti Dalfox, dirseaech, sqlmap dan lainnya melalui komunikai antar agen yang sinkron.

Penelitian difokuskan pada pengujian keamanan terhadap aplikasi web dummy yang memang telah disiapkan dengan kerentanan umum (seperti OWASP Juice Shop dan DVWA), sehingga tidak menimbulkan dampak keamanan pada sistem nyata. Hasil akhir dari penelitian ini berupa prototipe sistem pengujian otomatis dan laporan hasil deteksi kerentanan yang tervalidasi menggunakan metrik teknis dan usability.

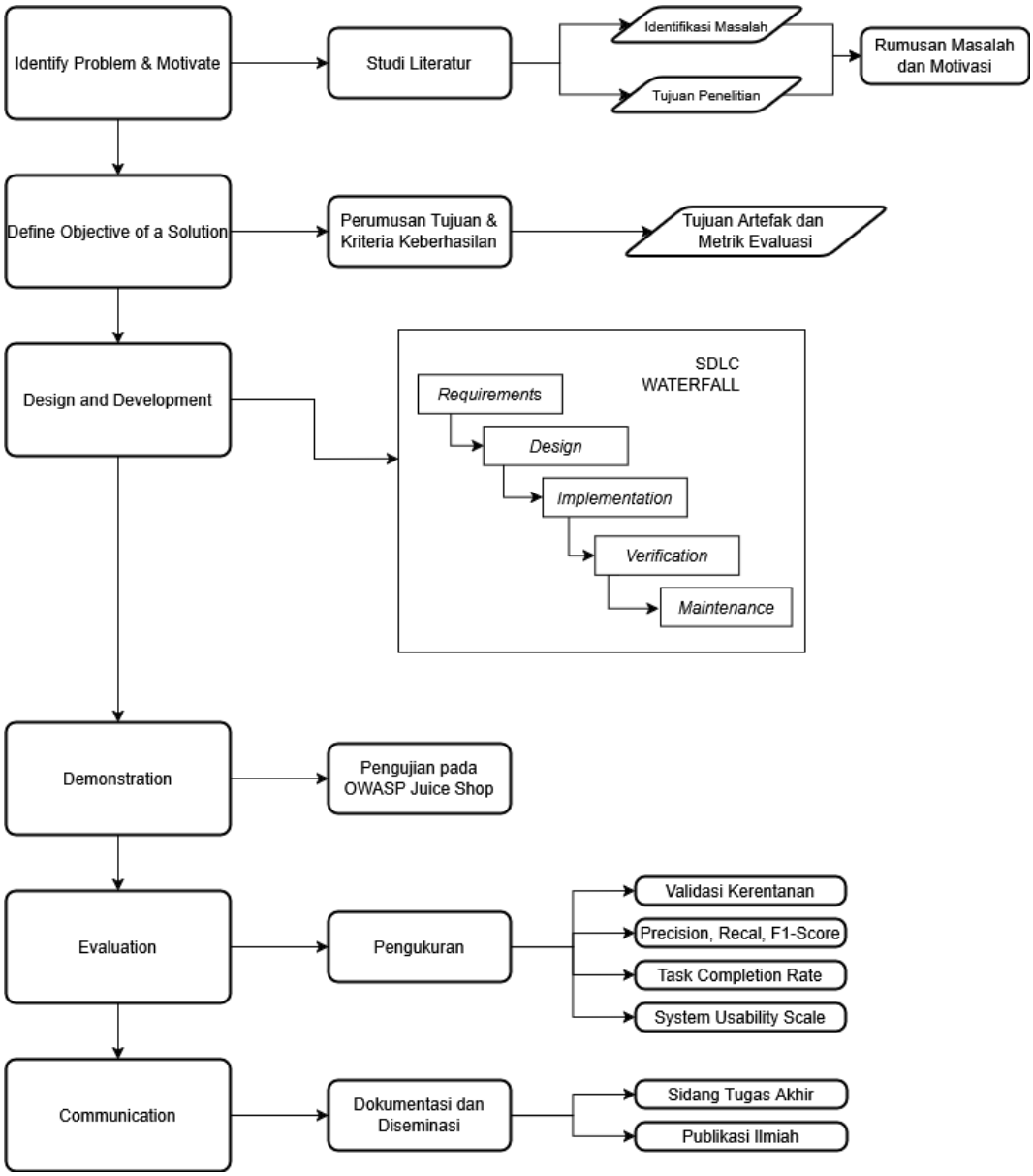
III.2 JENIS PENELITIAN

Penelitian ini menggunakan pendekatan mixed-method, yang menggabungkan metode kualitatif dan kuantitatif. Pendekatan kualitatif digunakan untuk memahami kebutuhan sistem, menganalisis proses kerja sistem multi-agen berbasi LLM dan MCP, serta menilai respon dan persepsi pengguna terhadap sistem yang dikembangkan. Sedangkan pendekatan kuantitatif digunakan untuk mengevaluas efektifitas dan kinerja sistem secara terukur. Evaluasi dilakukan untuk mengukur tingkat keberhasilan sistem dalam mendeteksi kerentanan dan persepsi pengguna dalam terhadap kemudahan penggunaan sistem

III.3 DESAIN PENELITIAN

Penelitian ini dirancang menggunakan pendekatan Design Science Research Methodology (DSRM) yang dikembangkan oleh Peffers et al.[49]. DSRM dipilih karena penelitian ini berorientasi pada penciptaan dan evaluasi artefak teknologi baru yang memiliki nilai ilmiah dan praktis, yaitu sistem multi-agen berbasis Large Language Model (LLM) dan Model Context Protocol (MCP) untuk pengujian keamanan aplikasi web berdasarkan OWASP Web Security Testing Guide (WSTG) v4.2.

Pendekatan ini memberikan panduan sistematis mulai dari identifikasi masalah hingga evaluasi solusi. Pada tahap Design and Development, model System Development Life Cycle (SDLC) dengan pendekatan Waterfall digunakan untuk mengatur proses rekayasa sistem secara teknis dan terstruktur.
Gambaran umum alur penelitian dapat dilihat pada diagram berikut:



III.3.1 Problem Identification and Motivation

Tahap pertama dalam metodologi DSRM adalah *Problem Identification and Motivation*. Tahap ini bertujuan untuk mengidentifikasi permasalahan penelitian secara spesifik dan memberikan justifikasi mengenai pentingnya permasalahan tersebut untuk diselesaikan.[49] Identifikasi masalah dilakukan melalui studi

literatur secara sistematis terhadap publikasi ilmiah, laporan industri, dan standar keamanan yang relevan seperti yang sudah dijelaskan pada subbab I.1 dan I.2.

III.3.2 Define the Objectives for a Solution

Tahap kedua dalam metodologi DSRM adalah *Define the Objectives for a Solution*. Tahap ini bertujuan untuk merumuskan tujuan dari artefak yang akan dikembangkan berdasarkan permasalahan yang telah diidentifikasi pada tahap sebelumnya.[49] Tujuan dari solusi ditentukan berdasarkan identifikasi masalah yang dijelaskan pada subbab I.4.

III.3.3 Design and Development

Tahap ini merupakan inti penelitian, di mana artefak (sistem multi-agen berbasis LLM dan MCP) dirancang dan dikembangkan. Untuk memastikan proses pengembangan berlangsung terstruktur, digunakan pendekatan System Development Life Cycle (SDLC) Waterfall, yang mencakup fase *Requirement*, *Design*, *Implementation*, *Verification*, dan *Maintenance*.

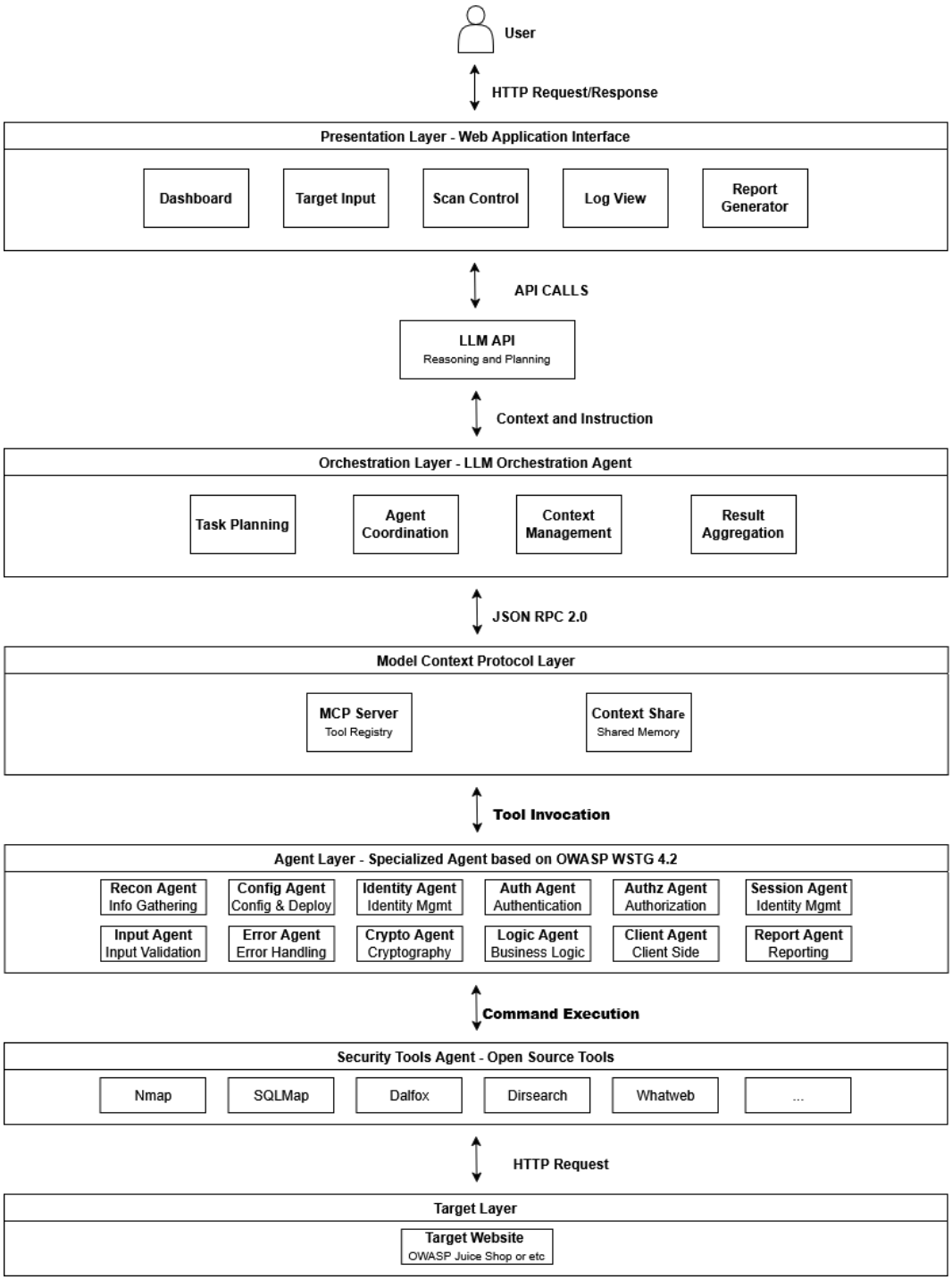
III.3.3.1 Requirement Analysis

Tahap *Requirement Analysis* bertujuan untuk mengidentifikasi kebutuhan sistem yang akan dikembangkan, baik dari sisi fungsionalitas, keamanan, maupun integrasi antarkomponen. Analisis kebutuhan dilakukan melalui studi literatur terhadap standar OWASP Web Security Testing Guide (WSTG) v4.2, observasi terhadap alat pengujian keamanan web yang telah ada seperti Feroxbuster, SQLmap, dan Dalfox, serta analisis kesenjangan (gap analysis) terhadap keterbatasan sistem konvensional yang masih bergantung pada intervensi manual. Hasil analisis menunjukkan bahwa sistem yang akan dikembangkan harus mampu melakukan pengujian keamanan aplikasi web secara otomatis berdasarkan kategori uji OWASP WSTG, mengintegrasikan komunikasi antaragen melalui Model Context Protocol (MCP), serta menghasilkan laporan hasil pengujian yang valid dan mudah dipahami pengguna. Selain itu, sistem juga perlu memenuhi kebutuhan non-fungsional seperti efisiensi waktu eksekusi, keamanan data, dan kemudahan integrasi dengan alat uji open-source.

III.3.3.2 System Design

Tahap *System Design* bertujuan untuk merancang arsitektur dan struktur logis sistem berdasarkan hasil analisis kebutuhan. Pada tahap ini, sistem dibangun dengan pendekatan multi-agen, di mana setiap agen bertanggung jawab terhadap satu bagian pengujian berdasarkan kategori OWASP WSTG 4.2. Sementara itu, lapisan antarmuka menyediakan Graphical User Interface (GUI) yang memungkinkan pengguna menjalankan pengujian, memantau proses, dan menerima laporan hasil secara otomatis. Arsitektur sistem dirancang dalam enam

layer yang saling terintegrasi untuk memisahkan concern dan memudahkan pengembangan serta pemeliharaan.
Gambar III.X menunjukkan arsitektur sistem secara keseluruhan.



III.3.3.3 Implementation

Tahap *Implementation* merupakan proses realisasi desain sistem menjadi bentuk yang dapat dijalankan secara fungsional. Seluruh modul yang telah dirancang pada

tahap sebelumnya mulai diimplementasikan dalam bentuk kode program dan integrasi sistem.Sistem dikembangkan menggunakan bahasa Python 3.11 dengan framework FastAPI untuk komunikasi antaragen dan pembuatan layanan backend. Komponen antarmuka pengguna dibangun menggunakan javascript, sedangkan pengelolaan komunikasi antaragen dilakukan melalui MCP Server yang menjadi pusat koordinasi antaragen berbasis konteks.
Pada tahap ini juga dilakukan integrasi dengan API LLM yang berfungsi sebagai mesin inferensi utama bagi setiap agen. Setiap agen dikembangkan untuk menjalankan tugas spesifik, seperti mendeteksi injeksi SQL, memeriksa konfigurasi yang tidak aman, atau menganalisis kebocoran data sensitif.

III.3.3.4 Testing

Tahap *Testing* dilakukan untuk memastikan bahwa artefak sistem yang dikembangkan berfungsi sesuai spesifikasi dan mampu mencapai tujuan penelitian. Pengujian dilakukan melalui beberapa tahapan mulai dari unit testing, integration testing, hingga system testing menggunakan aplikasi web dummy yakni OWASP Juice Shop. Pada tahap ini, diuji secara kolektif untuk menilai efektivitas koordinasi antaragen melalui MCP. Selain pengujian teknis, dilakukan juga evaluasi efektivitas sistem menggunakan metrik Task Completion Rate (TCR) untuk mengukur keberhasilan agen dalam menyelesaikan seluruh tahapan pengujian keamanan web. Selanjutnya, dilakukan pengujian usability menggunakan metode System Usability Scale (SUS) yang melibatkan praktisi keamanan siber bersertifikasi. Melalui kuesioner SUS, diperoleh penilaian terkait kemudahan penggunaan, efisiensi interaksi, dan tingkat kepuasan pengguna terhadap antarmuka dan kinerja sistem. Hasil dari tahap ini memberikan dasar untuk menilai efektivitas, reliabilitas, dan keterterimaan sistem sebagai artefak ilmiah.

III.3.3.5 Maintenance

Tahap terakhir dari siklus SDLC adalah *Maintenance*, yang dilakukan untuk memastikan sistem tetap berfungsi optimal setelah proses pengujian dan evaluasi selesai. Kegiatan pada tahap ini meliputi perbaikan kesalahan yang ditemukan selama uji coba, penyesuaian sistem terhadap pembaruan teknologi seperti versi terbaru tools pengujian, serta optimasi performa sistem untuk mencegah terjadinya masalah di masa mendatang. Selain itu, tahap ini juga mencakup peningkatan fitur sistem berdasarkan umpan balik pengguna dan hasil evaluasi usability. Melalui tahap pemeliharaan ini, sistem diharapkan tetap relevan, aman, dan dapat dikembangkan lebih lanjut sebagai dasar penelitian lanjutan di bidang keamanan siber berbasis kecerdasan buatan.

III.3.4 Demonstration

Tahap keempat dalam metodologi DSRM adalah *Demonstration*. Tahap ini bertujuan untuk mendemonstrasikan bahwa artefak yang dikembangkan dapat

menyelesaikan permasalahan yang telah diidentifikasi.[49] Demonstrasi dilakukan dengan menggunakan artefak untuk melakukan pengujian keamanan pada aplikasi web target dalam lingkungan yang terkontrol. Demonstrasi dilakukan pada lingkungan pengujian dengan spesifikasi sebagaimana disajikan pada Tabel III.X.

No	Komponen	Spesifikasi Minimum	Spesifikasi Rekomendasi	Keterangan
1	**Sistem Operasi**	Windows 10	Windows 11	Server/Desktop Edition
2	**Processor**	Intel Core i5 Gen 10 / AMD Ryzen 5 3600	Intel Core i7 Gen 10 / AMD Ryzen 7 3700X	Minimal 4 Cores
3	**RAM**	8 GB	16 GB	Untuk menjalankan multiple containers
4	**Storage**	50 GB SSD	100 GB SSD	Untuk logs dan hasil pengujian
5	**Docker**	v24.0+	v24.0+	Deployment target & isolasi
6	**Docker Compose**	v2.20+	v2.20+	Orchestration containers
7	**Python**	v3.10+	v3.11+	Runtime backend & MCP Server
8	**Node.js**	v18 LTS	v20 LTS	Runtime frontend
9	**Browser**	Chrome 120+	Chrome 120+	Akses antarmuka web
10	**Network**	Internet Connection	Internet Connection	Akses LLM API

Target pengujian yang digunakan adalah OWASP Juice Shop versi terbaru (Kimminich & OWASP Foundation, 2023). OWASP Juice Shop merupakan aplikasi web vulnerable yang dirancang khusus oleh OWASP Foundation untuk pembelajaran dan pengujian keamanan. Aplikasi ini mensimulasikan toko online dengan berbagai kerentanan keamanan yang sengaja ditanamkan.

III.3.5 Evaluation

Tahap kelima dalam metodologi DSRM adalah *Evaluation*. Tahap ini bertujuan untuk mengukur sejauh mana artefak yang dikembangkan mencapai tujuan yang telah ditetapkan pada tahap Define Objectives.[49] Evaluasi pada penelitian ini mencakup tiga aspek utama: evaluasi efektivitas deteksi, evaluasi fungsionalitas,

dan evaluasi usability. Tahap ini mengevaluasi efektivitas, efisiensi, dan tingkat penerimaan sistem. Evaluasi dilakukan melalui dua metode:

1. Pengujian Efektivitas Teknis

Mengukur tingkat keberhasilan sistem menyelesaikan tugas berdasarkan Task Completion Rate (TCR)

2. Evaluasi Usabilitas (System Usability Scale)

Melibatkan praktisi keamanan siber bersertifikasi untuk menilai kemudahan penggunaan, tampilan antarmuka, dan keandalan sistem melalui skala SUS.

Hasil evaluasi ini menjadi dasar perbaikan sistem dan validasi ilmiah atas artefak yang dikembangkan.

III.3.6 Communication

Tahap terakhir dalam metodologi DSRM adalah *Communication*, yaitu penyampaian hasil penelitian dan kontribusi ilmiah dari artefak yang dikembangkan kepada komunitas akademik dan praktisi profesional. Pada konteks penelitian ini, tahap komunikasi merupakan kegiatan Sidang Tugas Akhir, yang berfungsi sebagai sarana formal untuk mempresentasikan hasil penelitian, mendemonstrasikan artefak sistem multi-agen berbasis LLM dan MCP, serta menerima umpan balik dari dosen penguji sebagai bentuk validasi akademik. Dengan demikian, tahap Communication tidak hanya berfungsi sebagai dokumentasi, tetapi juga sebagai mekanisme validasi sosial dan ilmiah dari artefak yang dihasilkan melalui interaksi langsung dengan komunitas akademik.[49]

III.4 JADWAL PENELITIAN

No	Kegiatan	Waktu Pengerjaan																															
		2026																															
		Januari				Februari				Maret				April				Mei				Juni				Juli				Agustus			
		1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
	Bagian I: Penelitian																																
1	Studi Literatur																																
2																																	
3																																	
4																																	
5																																	
6																																	
7																																	
8	Penulisan BAB IV dan BAB V																																
	Bagian II: Pengumpulan, Seminar, dan Sidang Tugas Akhir																																
9	Pengumpulan Proposal TA																																
10	Seminar Penulisan TA																																
11	Pengumpulan 70% Dokumen TA																																
12	Seminar Hasil 70% Dokumen TA																																
13	Pengumpulan 100% Dokumen TA																																
14	Simulasi Sidang TA																																
15	Sidang Tugas Akhir																																
16	Revisi Tugas Akhir																																
17	Pengumpulan Revisi Tugas Akhir																																

Keterangan

: Pengerjaan

: Deadline

DAFTAR PUSTAKA

[1] BSSN, "Lanskap Keamanan Siber Indonesia 2024," Jan. 2025.

[2] Hackerone, "8th Annual Hacker-Powered Security Report 2024-2025," Nov. 2024.

[3] ISACA, "State of Cybersecurity 2024," Oct. 2024.

[4] K. Božić, N. Penevski, and S. Adamović, "Penetration Testing and Vulnerability Assessment: Introduction, Phases, Tools and Methods," *Sinteza 2019 - International Scientific Conference on Information Technology and Data Related Research*, pp. 229–234, May 2019, doi: 10.15308/SINTEZA-2019-229-234.

[5] K. Abdulghaffar, N. Elmrabit, and M. Yousefi, "Enhancing Web Application Security through Automated Penetration Testing with Multiple Vulnerability Scanners," *Computers 2023, Vol. 12, Page 235*, vol. 12, no. 11, p. 235, Nov. 2023, doi: 10.3390/COMPUTERS12110235.

[6] A. S. B. Singh, Y. Yusof, and Y. Nathan, "EAGLE: GUI-Based Penetration Testing Tool for Scanning and Enumeration," in *Proceedings - International Conference on Developments in eSystems Engineering, DeSE*, Institute of Electrical and Electronics Engineers Inc., 2021, pp. 97–101. doi: 10.1109/DESE54285.2021.9719338.

[7] D. Goyal, S. Subramanian, A. Peela, and N. P. Shetty, "Hacking, The Lazy Way: LLM Augmented Pentesting," Sep. 2024.

[8] G. Deng *et al.*, "PentestGPT: An LLM-empowered Automatic Penetration Testing Tool," Jun. 2024, [Online]. Available: http://arxiv.org/abs/2308.06782

[9] L. Muzsai, D. Imolai, and A. Lukács, "HackSynth: LLM Agent and Evaluation Framework for Autonomous Penetration Testing," 2024, [Online]. Available: http://arxiv.org/abs/2412.01778

[10] S. G. Bianou and R. G. Batogna, "PENTEST-AI, an LLM-Powered Multi-Agents Framework for Penetration Testing Automation Leveraging Mitre Attack," *Proceedings of the 2024 IEEE International Conference on Cyber Security and Resilience, CSR 2024*, pp. 763–770, 2024, doi: 10.1109/CSR61664.2024.10679480.

[11] OWASP Foundation, "OWASP WSTG - v4.2." Accessed: Jun. 28, 2025. [Online]. Available: https://owasp.org/www-project-web-security-testing-guide/v42/

[12] P. Wake, S. Black, and J. Young, "Work in Progress: Evaluation of Security Standards through a Cyber Range using Hackers' Tactics, Techniques and Procedures," in *Proceedings - 8th IEEE European Symposium on Security and Privacy Workshops, Euro S and PW 2023*, Institute of Electrical and

Electronics Engineers Inc., 2023, pp. 653–658. doi: 10.1109/EuroSPW59978.2023.00076.

[13] Anthropic, "Introducing the Model Context Protocol," https://www.anthropic.com/news/model-context-protocol. Accessed: Jun. 15, 2025. [Online]. Available: https://www.anthropic.com/news/model-context-protocol

[14] M. Izzat, F. A. Saputra, and I. Syarif, "Design and Implementation of Distributed Web Application Vulnerability Assessment Tools for Securing Complex Microservices Environment," *International Journal of Safety and Security Engineering*, vol. 15, no. 2, pp. 267–273, Feb. 2025, doi: 10.18280/ijsse.150207.

[15] A. E. Hafez and M. M. Almustafa, "Detecting Security Vulnerabilities in Web Applications: A Proposed System," *International Journal of Safety and Security Engineering*, vol. 14, no. 6, pp. 1933–1940, Dec. 2024, doi: 10.18280/ijsse.140627.

[16] "OWASP Top Ten Web Application Security Risks | OWASP Foundation." Accessed: Jan. 12, 2026. [Online]. Available: https://owasp.org/www-project-top-ten/

[17] "OWASP Application Security Verification Standard (ASVS) | OWASP Foundation." Accessed: Jan. 12, 2026. [Online]. Available: https://owasp.org/www-project-application-security-verification-standard/

[18] BSSN, "Lanskap Keamanan Siber Indonesia 2024," Jan. 2025. Accessed: Jun. 15, 2025. [Online]. Available: https://ilmubersama.com/2025/03/30/lanskap-keamanan-siber-indonesia-2024-bssn/

[19] "OWASP Top 10:2025." Accessed: Jan. 12, 2026. [Online]. Available: https://owasp.org/Top10/2025/

[20] M. Bishop, "About penetration testing," *IEEE Secur Priv*, vol. 5, no. 6, pp. 84–87, Nov. 2007, doi: 10.1109/MSP.2007.159.

[21] NIST SP 800-53, "Security and Privacy Controls for Information Systems and Organizations," Dec. 2020, doi: 10.6028/NIST.SP.800-53R5.

[22] K. Božić, N. Penevski, and S. Adamović, "Penetration Testing and Vulnerability Assessment: Introduction, Phases, Tools and Methods," *Sinteza 2019 - International Scientific Conference on Information Technology and Data Related Research*, pp. 229–234, May 2019, doi: 10.15308/SINTEZA-2019-229-234.

[23] F. Barman, N. Alkaabi, H. Almenhali, M. Alshedi, and R. Ikuesan, "A Methodical Framework for Conducting Reconnaissance and Enumeration in the Ethical Hacking Lifecycle," *European Conference on Cyber Warfare and Security*, vol. 22, no. 1, pp. 54–64, Jun. 2023, doi: 10.34190/ECCWS.22.1.1438.

[24] D. Kongara and S. Krishnama, "A Process of Penetration Testing Using Various Tools," *Mesopotamian Journal of CyberSecurity*, vol. 2023, pp. 93–103, Dec. 2023, doi: 10.58496/MJCS/2023/014.

[25] Z. Bin Akhtar and A. T. Rawol, "Uncovering Cybersecurity Vulnerabilities: A Kali Linux Investigative Exploration Perspective," *International Journal of Advanced Network, Monitoring and Controls*, vol. 9, no. 2, pp. 11–22, Jun. 2024, doi: 10.2478/IJANMC-2024-0012.

[26] Dr. C. Gomathy, Dr. V. Geetha, Mr. P. V. Sri Ram, and S. P. L N, "NOVEL STUDY ON ETHICAL HACKING AND VULNERABILITY TESTING," *INTERANTIONAL JOURNAL OF SCIENTIFIC RESEARCH IN ENGINEERING AND MANAGEMENT*, vol. 08, no. 09, pp. 1–6, Sep. 2024, doi: 10.55041/IJSREM37510.

[27] A. A. Alghamdi, "Effective penetration testing report writing," *International Conference on Electrical, Computer, Communications and Mechatronics Engineering, ICECCME 2021*, Oct. 2021, doi: 10.1109/ICECCME52200.2021.9591097.

[28] Y. Hu, W. Wang, S. Khurshid, and M. Tiwari, "Efficient IAM Greybox Penetration Testing," Feb. 2025, [Online]. Available: http://arxiv.org/abs/2304.14540

[29] S. Mehta, G. Raj, and D. Singh, "Penetration testing as a test phase in web service testing a black box pen testing approach," *Smart Innovation, Systems and Technologies*, vol. 78, pp. 623–635, 2018, doi: 10.1007/978-981-10-5547-8_64.

[30] A. Maspupah, "LITERATURE REVIEW: ADVANTAGES AND DISADVANTAGES OF BLACK BOX AND WHITE BOX TESTING METHODS," *Jurnal Techno Nusa Mandiri*, vol. 21, no. 2, pp. 151–162, Sep. 2024, doi: 10.33480/TECHNO.V21I2.5776.

[31] A. Kumar, "Penetration Testing Tools and Techniques," *https://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/978-1-6684-8218-6.ch012*, pp. 280–306, Jan. 1AD, doi: 10.4018/978-1-6684-8218-6.CH012.

[32] "Internet use in 2024 — DataReportal – Global Digital Insights." Accessed: Aug. 24, 2025. [Online]. Available: https://datareportal.com/reports/digital-2024-deep-dive-the-state-of-internet-adoption

[33] "urbanadventurer/WhatWeb: Next generation web scanner." Accessed: Dec. 31, 2025. [Online]. Available: https://github.com/urbanadventurer/WhatWeb

[34] "sqlmap: automatic SQL injection and database takeover tool." Accessed: Dec. 31, 2025. [Online]. Available: https://sqlmap.org/

[35] M. Maulana, S. Ambo, and R. Mujastuti, "Introduction to Web Security: Detecting XSS with Dalfox and Paramspider," Nov. 2024. [Online]. Available: https://www.researchgate.net/publication/386148513

[36] F. Mohammed, N. A. A. Rahman, Y. Yusof, and J. Juremi, "Automated Nmap Toolkit," *ASSIC 2022 - Proceedings: International Conference on Advancements in Smart, Secure and Intelligent Computing*, 2022, doi: 10.1109/ASSIC55218.2022.10088375.

[37] "whois(1) — Arch manual pages." Accessed: Dec. 31, 2025. [Online]. Available: https://man.archlinux.org/man/whois.1.en

[38] "testssl/testssl.sh: Testing TLS/SSL encryption anywhere on any port." Accessed: Dec. 31, 2025. [Online]. Available: https://github.com/testssl/testssl.sh

[39] "Nuclei Overview - ProjectDiscovery Documentation." Accessed: Aug. 24, 2025. [Online]. Available: https://docs.projectdiscovery.io/opensource/nuclei/overview

[40] "Puppeteer | Puppeteer." Accessed: Jan. 01, 2026. [Online]. Available: https://pptr.dev/

[41] A. Vaswani *et al.*, "Attention Is All You Need."

[42] H. Naveed *et al.*, "A Comprehensive Overview of Large Language Models," *ACM Trans Intell Syst Technol*, vol. 16, no. 5, Aug. 2025, doi: 10.1145/3744746.

[43] M. Naseriparsa *et al.*, "Survey and analysis of hallucinations in large language models: attribution to prompting strategies or model behavior," *Front Artif Intell*, vol. 8, p. 1622292, 2025, doi: 10.3389/FRAI.2025.1622292.

[44] M. Wooldridge, "An Introduction to MultiAgent Systems - 2nd Edition," *ACM SIGACT News*, vol. 41, no. 1, p. 462, 2009, Accessed: Jan. 13, 2026. [Online]. Available: https://www.wiley.com/en-us/An+Introduction+to+MultiAgent+Systems%2C+2nd+Edition-p-9780470519462

[45] T. Guo *et al.*, "Large Language Model Based Multi-agents: A Survey of Progress and Challenges," vol. 9, pp. 8048–8057, Aug. 2024, doi: 10.24963/IJCAI.2024/890.

[46] X. Li, S. Wang, S. Zeng, Y. Wu, and Y. Yang, "A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges," *Vicinagearth 2024 1:1*, vol. 1, no. 1, pp. 9-, Oct. 2024, doi: 10.1007/S44336-024-00009-2.

[47] "Specification - Model Context Protocol." Accessed: Jan. 13, 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2025-03-26

[48] V. M. , JR. FONT, *ULTIMATE GUIDE TO THE SDLC*. FONTLIFE PUBLICATION, LLC, 2012.

[49] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, Dec. 2007, doi: 10.2753/MIS0742-1222240302.

[50] O. Vechtomova, “Introduction to Information Retrieval,” *Computational Linguistics*, vol. 35, no. 2, pp. 307–309, Jun. 2009, doi: 10.1162/COLI.2009.35.2.307.

[51] J. Brooke, “SUS: A ‘Quick and Dirty’ Usability Scale,” *Usability Evaluation In Industry*, pp. 207–212, Jun. 1996, doi: 10.1201/9781498710411-35.

[52] A. Bangor, P. T. Kortum, and J. T. Miller, “An Empirical Evaluation of the System Usability Scale,” *International journal of human computer interactions*, vol. 24, no. 6, pp. 574–594, Aug. 2008, doi: 10.1080/10447310802205776.

[53] G. Deng *et al.*, “PentestGPT: An LLM-empowered Automatic Penetration Testing Tool,” Jun. 2024, Accessed: Oct. 16, 2025. [Online]. Available: http://arxiv.org/abs/2308.06782

[54] K. Vadla, K. G. Teja, S. Amanuddin, A. Bhatta, and M. Rajeshwar, “Integrating Penetration Testing Tools into a Website,” in *2024 15th International Conference on Computing Communication and Networking Technologies, ICCCNT 2024*, Institute of Electrical and Electronics Engineers Inc., 2024. doi: 10.1109/ICCCNT61001.2024.10724706.

[55] L. Muzsai, D. Imolai, and A. Lukács, “HackSynth: LLM Agent and Evaluation Framework for Autonomous Penetration Testing,” Dec. 2024, [Online]. Available: http://arxiv.org/abs/2412.01778

[56] V. Mayoral-Vilches *et al.*, “CAI: An Open, Bug Bounty-Ready Cybersecurity AI,” Apr. 2025, Accessed: Jan. 13, 2026. [Online]. Available: https://arxiv.org/abs/2504.06017v2

DAFTAR RIWAYAT HIDUP

Nama	: Martua Raja Doli Pangaribuan	
Tempat/tanggal lahir	: P. Siantar / 8 Februari 2004	
Riwayat pendidikan	: 1. TK Santa Maria	2010
	2. SDS RK Budi Mulia 3	2016
	3. SMPS RK Bintang Timur	2019
	4. SMAS RK Budi Mulia	2022